

MODULE 1

Professional Software Development

- Professional software development means developing software for business, organizations, or customers rather than for personal or hobby use.
- It focuses on building software that is reliable, maintainable, and suitable for long-term use.
- Professional software is:
 - Used by people other than the developer.
 - Usually developed by teams.
 - Maintained and updated throughout its lifetime.
- Software engineering supports professional software development by covering:
 - Software specification (defining requirements)
 - Software design and implementation
 - Software validation (testing)
 - Software evolution (modifying after release)
- Software includes:
 - Computer programs
 - Documentation
 - Libraries
 - Configuration data
 - Support resources such as websites
- Two types of software products:
 - **Generic products**
 - Developed for the open market.
 - Sold to any customer.
 - Specification is controlled by the developer.

- Examples: word processors, graphics software, ERP systems.
- **Customized (bespoke) products**
 - Developed for a specific customer.
 - Specification is controlled by the customer.
 - Examples: embedded systems, air traffic control software, business systems.
- Essential attributes of good software:
 - **Maintainability**
 - Easy to modify and update as needs change.
 - **Dependability and Security**
 - Reliable, safe, and protected from unauthorized access.
 - **Efficiency**
 - Uses memory, processor time, and other resources effectively.
 - **Acceptability**
 - Easy to understand and use, and compatible with other systems.
- Software engineering is an engineering discipline concerned with all aspects of software production.
- Fundamental software engineering activities:
 - Software specification
 - Software development
 - Software validation
 - Software evolution
- Difference between software engineering and computer science:
 - Computer science focuses on theory and fundamentals.
 - Software engineering focuses on practical development and delivery of useful software.
- Difference between software engineering and system engineering:
 - System engineering covers hardware, software, and overall system development.

- Software engineering deals specifically with software and is a part of system engineering.
 - Key challenges in software engineering:
 - Handling increasing diversity of systems.
 - Reducing delivery time.
 - Developing trustworthy and secure software.
 - Software costs:
 - Software often costs more than hardware.
 - Maintenance usually costs more than initial development.
 - Software engineering aims for cost-effective development.
 - Software project failures often happen because:
 - Systems become increasingly complex.
 - Organizations do not follow proper software engineering methods, leading to expensive and unreliable software.
-

Essential Attributes of Good Software

- Good software should have four essential attributes:
 - Maintainability
 - Dependability and Security
 - Efficiency
 - Acceptability
- **Maintainability**
 - Software should be easy to modify and update.
 - Changes may be needed to fix bugs, add features, or adapt to new requirements.
 - Maintainability is important because business needs change over time.
 - **Example:** A banking system should be easy to update when new financial regulations are introduced.
- **Dependability and Security**

- Software should be reliable, safe, and secure.
 - It should function correctly without failures.
 - It should protect data from unauthorized access and cyber attacks.
 - Users should be able to trust the software.
 - **Example:** An online banking system must securely protect customer accounts and transactions.
 - **Efficiency**
 - Software should make effective use of system resources such as memory, processor time, and storage.
 - It should provide fast response times and good performance, even under heavy load.
 - Efficiency is especially important in real-time and large-scale systems.
 - **Example:** A traffic control system must process data quickly to manage vehicle flow.
 - **Acceptability**
 - Software should be understandable and user-friendly.
 - It should be easy to learn and use.
 - It should be compatible with other systems and fit the users' working environment.
 - Good usability improves user satisfaction and productivity.
 - **Example:** An online learning platform should have a simple interface that students can navigate easily.
-

Key Challenges Affecting Software

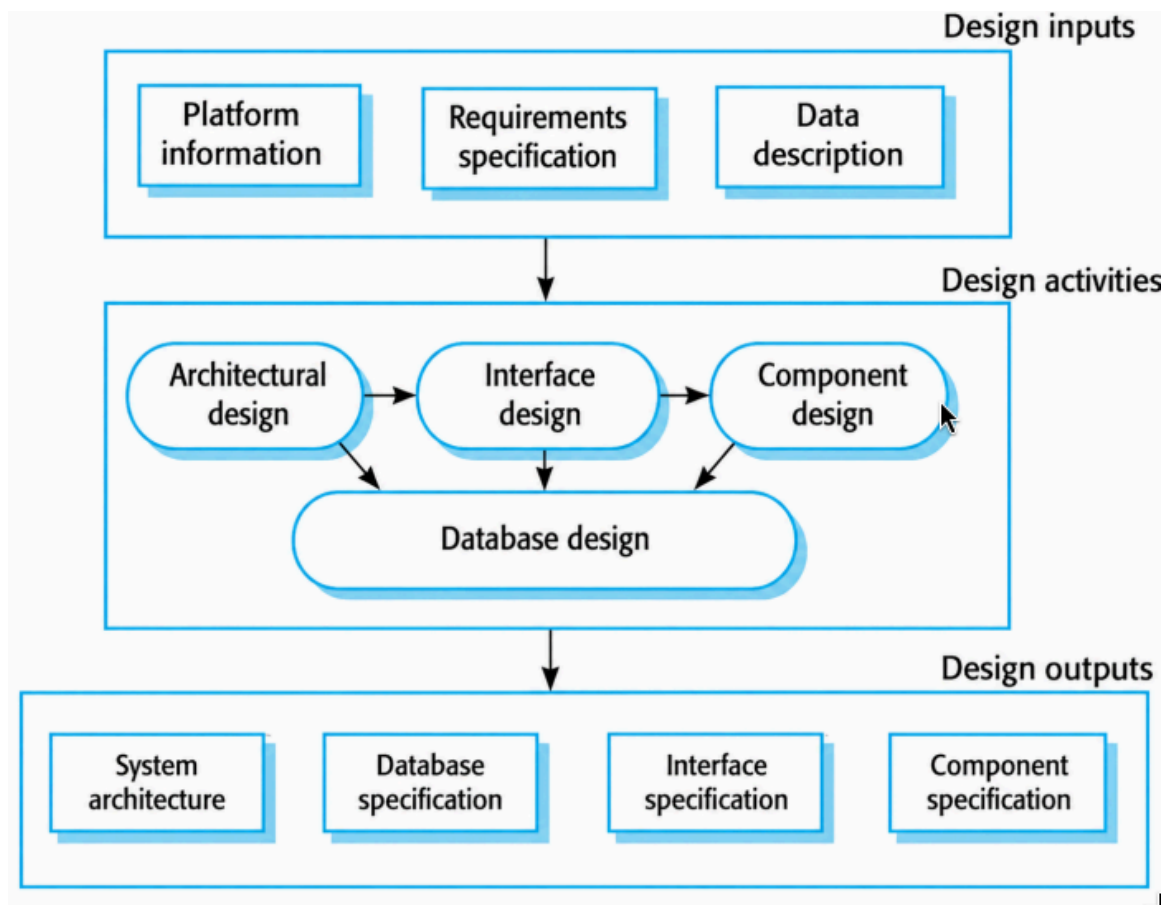
- Modern software development faces several major challenges:
 - Heterogeneity
 - Business and Social Change
 - Security and Trust
 - Scale

- **Heterogeneity**
 - Software must work across different hardware platforms and operating systems.
 - It should support desktops, mobile devices, embedded systems, and cloud environments.
 - Modern systems often need to interact with legacy systems.
 - Ensuring compatibility and smooth communication between different systems is a major challenge.
 - **Business and Social Change**
 - Business needs and social environments change rapidly due to globalization and new technologies.
 - Software must adapt quickly to these changing requirements.
 - It should be flexible and easy to modify.
 - Frequent updates and rapid development are often necessary.
 - **Security and Trust**
 - Software stores sensitive personal and organizational data.
 - It must protect this data from cyber attacks and unauthorized access.
 - Users should be able to trust that the software is safe, secure, and reliable.
 - Security failures can lead to financial loss and privacy issues.
 - **Scale**
 - Software systems range from small embedded applications to large cloud-based systems serving millions of users.
 - Large-scale systems are more complex to develop and maintain.
 - They must handle challenges such as performance, scalability, and maintenance effectively.
-

General Model of the Software Design Process

- Software design is the process of converting software requirements into a design that can be implemented as a software system.

- The design process develops the architecture, interfaces, components, and data structures of the system.



- **Activities in the Design Process:**

- **Architectural Design**

- Design the overall structure of the system.
 - Identify major components and their relationships.
 - Divide the system into subsystems.
 - **Output:** System architecture.

- **Interface Design**

- Design interfaces between system components.
 - Specify user interfaces and communication interfaces.
 - Define how components interact with each other.
 - **Output:** Interface specifications.

- **Component Design**
 - Design each component in detail.
 - Develop algorithms and processing logic.
 - Specify the internal structure of modules.
 - **Output:** Component specifications.
 - **Database Design**
 - Design data structures and the database schema.
 - Define relationships among data items.
 - Specify storage and retrieval mechanisms.
 - **Output:** Database design specification.
 - **Design Inputs**
 - Software Requirements Specification (SRS)
 - Functional Requirements
 - Non-Functional Requirements
 - System Constraints
 - **Design Outputs**
 - Architectural Design
 - Interface Specifications
 - Component Design Documents
 - Database Design
 - Complete Software Design Specification
-

Software Engineering Ethics

- Software engineering ethics refers to the moral and professional responsibilities that software engineers must follow while developing and maintaining software.
- Software engineers must:
 - Act honestly and responsibly.
 - Consider the impact of their work on society.

- Follow legal and professional standards.
- Key ethical responsibilities:
 - **Confidentiality**
 - Respect the privacy of employers and clients.
 - Do not disclose confidential information without permission.
 - **Competence**
 - Accept only work that matches your skills and experience.
 - Do not misrepresent your abilities.
 - **Intellectual Property Rights**
 - Respect copyrights, patents, licenses, and ownership of software and data.
 - **Computer Misuse**
 - Do not misuse computer systems.
 - Avoid activities such as unauthorized access, malware creation, or illegal use of resources.
- Professional organizations such as **ACM** and **IEEE** provide codes of ethics and standards for software engineers.
- The ACM/IEEE Software Engineering Code of Ethics is based on eight principles:
 - Public – Act in the public interest.
 - Client and Employer – Act in their best interests while protecting the public.
 - Product – Ensure high professional standards in software products.
 - Judgment – Maintain honesty and independent judgment.
 - Management – Promote ethical management practices.
 - Profession – Improve the integrity and reputation of the profession.
 - Colleagues – Be fair, supportive, and respectful to coworkers.
 - Self – Continue learning and practice ethical behavior throughout your career.
- Common ethical dilemmas:
 - Balancing confidentiality with public safety.

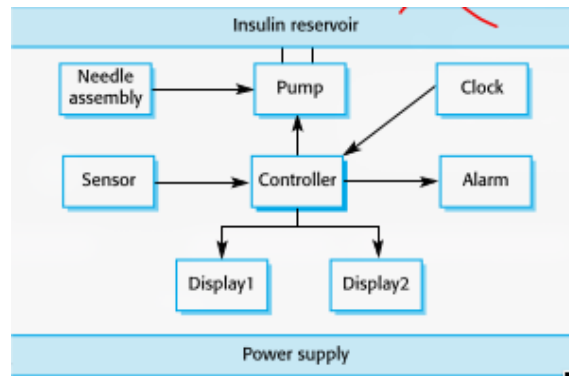
- Deciding whether to report project problems.
 - Handling disagreements with management decisions.
 - Dealing with unethical practices such as falsifying safety results.
 - In extreme situations where public safety is at serious risk, disclosure of information may be justified, but it should be considered only as a last resort.
 - Ethics are important because software systems affect many areas of life, and engineers have a responsibility to develop safe, reliable, and trustworthy systems.
-

Case Studies

- Case studies are used to explain software engineering concepts using real-world systems.
- Different case studies are used because software engineering practices vary depending on the type of system being developed.

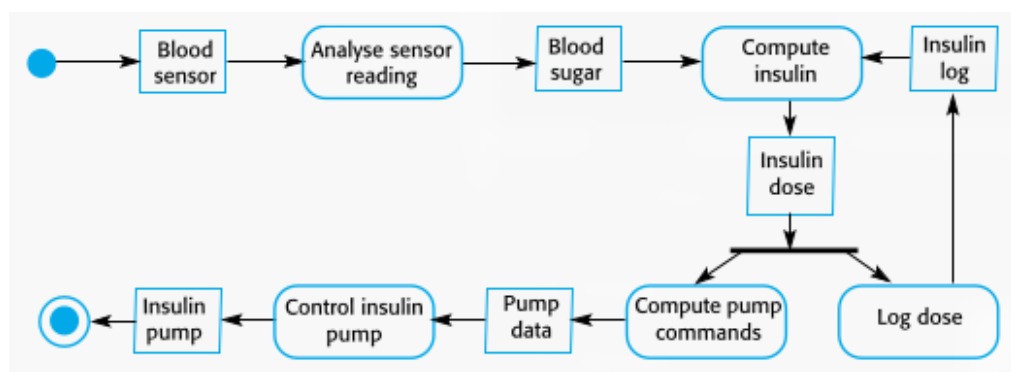
Insulin Pump

- An **insulin pump** is a medical embedded system that acts like the pancreas.
- It collects data from a blood sensor and automatically delivers insulin to the patient.
- It is mainly used by people with **diabetes**, where the body does not produce enough insulin.
- Insulin helps regulate blood glucose (sugar) levels.
- Traditional treatment involves:
 - Measuring blood sugar periodically.
 - Injecting insulin manually.



- **Activity Model (Working)**

- A microsensor continuously measures blood sugar-related data.
- The system analyzes the sensor reading.
- The controller calculates the required insulin dose based on:
 - Current blood sugar level.
 - Previous insulin injections.
- Control commands are generated for the pump.
- The pump delivers the calculated insulin dose.
- The delivered dose is recorded in a log.
- The process repeats continuously to maintain normal glucose levels.



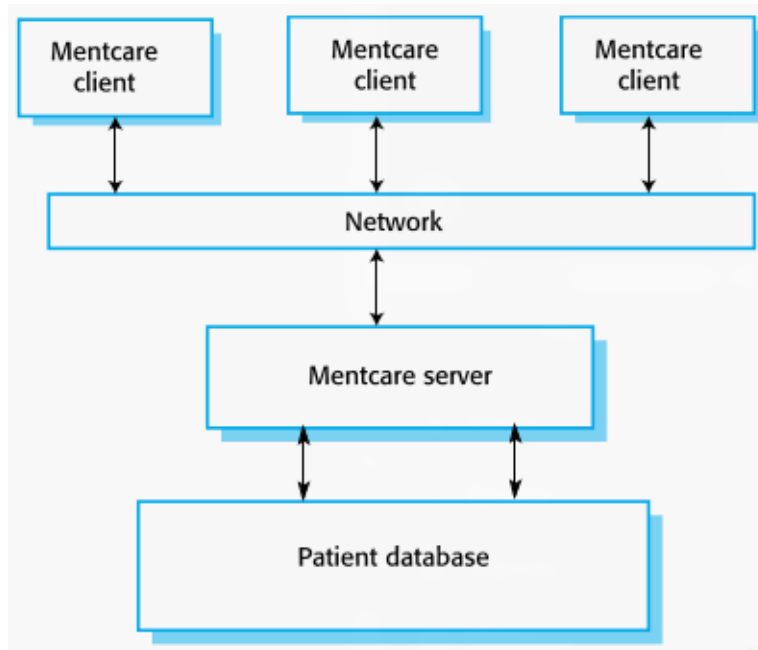
- **Main Components**

- **Sensor** – Continuously measures blood sugar level.
- **Controller** – Processes sensor data and calculates the insulin dose.
- **Pump** – Delivers the calculated amount of insulin.
- **Needle Assembly** – Injects insulin into the body.

- **Clock** – Controls the timing of measurements and insulin delivery.
- **Alarm System** – Alerts the patient about faults or abnormal conditions.
- **Display Units** – Show system status and insulin information.
- **Incorrect dosing can cause:**
 - **Too little blood sugar (Hypoglycemia):** unconsciousness or death.
 - **Too much blood sugar (Hyperglycemia):** long-term damage to organs such as the eyes, kidneys, and heart.
- **Essential high-level requirements**
 - **Safety** – Prevent overdose or underdose of insulin.
 - **Reliability** – Deliver the correct insulin dose consistently.
 - **Availability** – Be operational whenever insulin is needed.
 - **Accuracy** – Sensor readings and dose calculations must be precise.
 - **Fault Detection** – Detect failures and trigger alarms.
 - **Continuous Monitoring** – Automatically check blood sugar at regular intervals.
- **Key characteristics**
 - Safety-critical system.
 - Must always be available when required.
 - Must deliver the correct insulin dose reliably.
 - Failure can lead to serious health consequences, so strict safety and reliability are essential.

Mentcare System

- **Mentcare** is a patient information system designed to support mental health clinics.
- It helps doctors, nurses, administrators, and other staff manage patient information and treatment effectively.
- It stores patient records and provides management information for healthcare organizations.



- **Basic concepts**

- Uses a centralized database to store patient records.
- Can run locally on laptops when there is no network connection.
- Synchronizes local data with the central database when connectivity is restored.
- Can exchange information with other clinical information systems.
- Supports both medical and administrative activities.

- **Goals of the Mentcare system**

- Provide management information for planning, monitoring, and performance assessment.
- Support doctors, nurses, and other medical staff with timely and accurate patient information.
- Improve patient care by maintaining complete treatment histories and records.
- Increase efficiency by reducing paperwork and improving communication among staff.

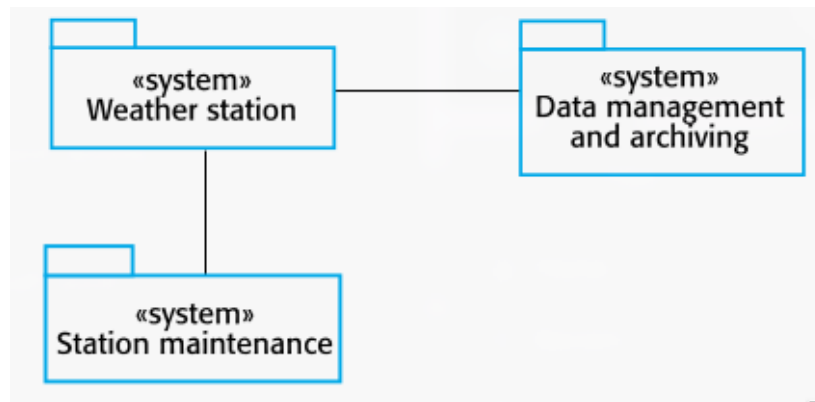
- **Users of Mentcare**

- Clinical staff:
 - Doctors

- Nurses
 - Health visitors
- Non-medical staff:
 - Receptionists
 - Records staff
 - Administrators
- **Key features**
 - Individual care management with detailed patient records, summaries, and treatment plans.
 - Patient monitoring to track treatment progress and generate alerts for problems.
 - Administrative reporting with detailed and anonymized reports for management.
 - Data synchronization between local devices and the central database.
 - Integration with other clinical information systems.
 - Centralized database management for consistent and organized patient records.
- **Challenges and constraints**
 - Patients may miss appointments or lose medication.
 - Some patients may behave unpredictably.
 - Some patients may be a danger to themselves or others.
 - The system must support legal procedures such as compulsory detention (sectioning) when required.
- **Critical requirements**
 - **Privacy and Confidentiality** – Protect patient information from unauthorized access.
 - **Safety** – Support safe treatment and care of patients.
 - **Availability** – Ensure patient records are accessible whenever healthcare staff need them.

Wilderness Weather Station

- A **Wilderness Weather Station** is a data collection system that gathers weather information from remote and inaccessible locations.
- It is used by governments or organizations to monitor environmental conditions in large wilderness areas.
- The station measures:
 - Temperature
 - Air pressure
 - Sunshine
 - Rainfall
 - Wind speed
 - Wind direction
- Each measuring instrument is controlled by software that:
 - Takes readings at regular intervals.
 - Manages the collected data.
- **Main system components**
 - **Weather Station System**
 - Collects weather data.
 - Performs initial data processing.
 - Transmits data to the data management system.
 - **Data Management and Archiving System**
 - Collects data from all weather stations.
 - Processes, analyzes, and stores the data.
 - **Station Maintenance System**
 - Monitors the health of weather stations.
 - Communicates with stations through satellite.
 - Reports faults and problems.



- **Station characteristics**
 - Battery-powered and self-contained.
 - Communicates using satellite links.
 - Operates in harsh environmental conditions.
 - Can function without constant human supervision.
- **Software responsibilities**
 - Collect and aggregate weather data.
 - Monitor instruments, power supply, and communication hardware.
 - Detect and report faults.
 - Manage power efficiently by charging batteries when possible and shutting down generators in dangerous weather.
 - Support dynamic reconfiguration by updating software and switching to backup instruments when failures occur.
- Although collecting weather data is straightforward, the overall software is complex because it must ensure:
 - High reliability.
 - Autonomous operation.
 - Fault tolerance.
 - Continuous operation in remote and harsh environments.

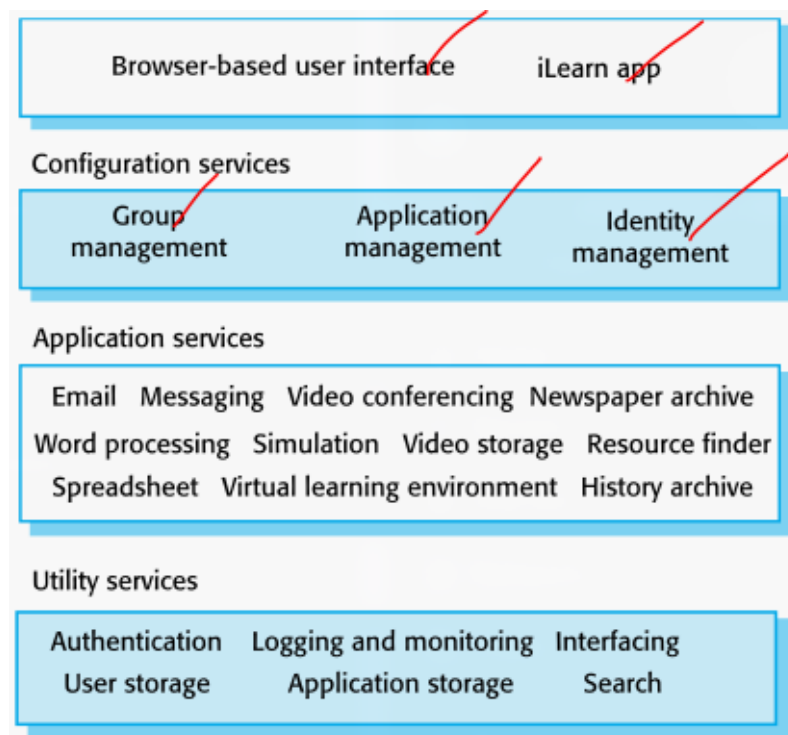
Digital Learning Environment

- A **Digital Learning Environment (DLE)** is a framework that combines general-purpose and educational software tools to support teaching and learning in

schools.

- It is designed to:
 - Support students of different age groups.
 - Provide online learning resources and applications.
 - Allow teachers and learners to choose tools based on their needs.
- Common tools included in a DLE:
 - Virtual Learning Environment (VLE)
 - Email services
 - Homework submission and assessment tools
 - Games and simulations
 - Spreadsheets
 - Educational content and learning resources
- **Key characteristics**
 - Distributed and service-oriented system.
 - Components are accessible through the Internet.
 - Not all tools need to be located in one place.
 - Services can be replaced or updated as new ones become available.
- **Types of services**
 - **Utility services**
 - Provide basic functions such as authentication, logging, storage, and search.
 - **Application services**
 - Provide applications like email, conferencing, simulations, and learning content.
 - **Configuration services**
 - Customize the environment and define how services are shared among students, teachers, and parents.
- **Integrated services**

- Provide APIs for direct communication between services.
- Allow automatic service-to-service interaction.
- **Independent services**
 - Accessed through a web browser.
 - Operate separately from other services.
 - Information sharing requires user actions such as copy-paste or re-authentication.
- **Advantages**
 - Flexible and easy to customize.
 - Supports different learning needs and age groups.
 - New services can be added or existing ones replaced without redesigning the entire system.
 - Encourages reuse of software components and online services.

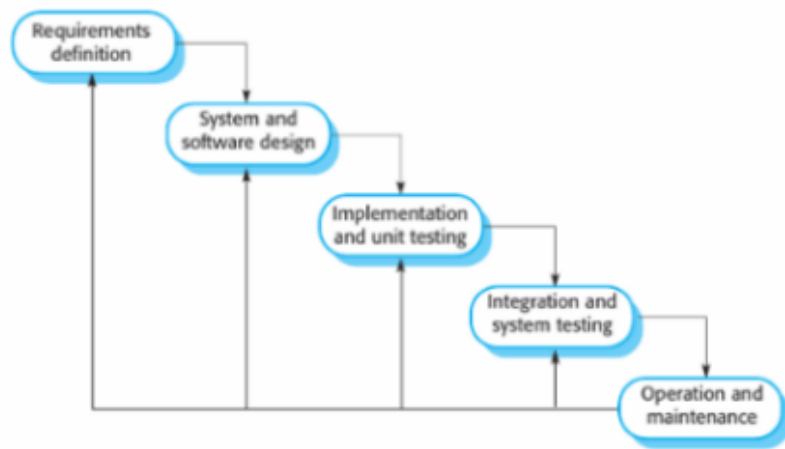


Software Process Models

- A **software process model** is an abstract representation of the activities involved in developing a software system.
- It describes how software is planned, developed, tested, and maintained.
- All software processes generally include four main activities:
 - **Specification** – Define what the system should do.
 - **Design and Implementation** – Design the system and write the code.
 - **Validation** – Check that the software meets customer requirements.
 - **Evolution** – Modify the software to meet changing needs after deployment.
- There are **three main types of software process models**:
 - Waterfall Model
 - Incremental Development Model
 - Integration and Configuration (Reuse-Oriented) Model

Waterfall Model

- The Waterfall Model is a **plan-driven** software development model.
- Development is divided into separate and distinct phases.
- Each phase is completed before moving to the next phase.
- It is most suitable when requirements are well understood and changes are limited.
- **Stages / Working:**
 - Requirements analysis and definition
 - System and software design
 - Implementation and unit testing
 - Integration and system testing
 - Operation and maintenance



- **Advantages:**

- Simple and easy to understand.
- Clearly defined phases and deliverables.
- Good documentation throughout development.
- Suitable for projects with stable requirements.
- Helps coordinate work in large system engineering projects.

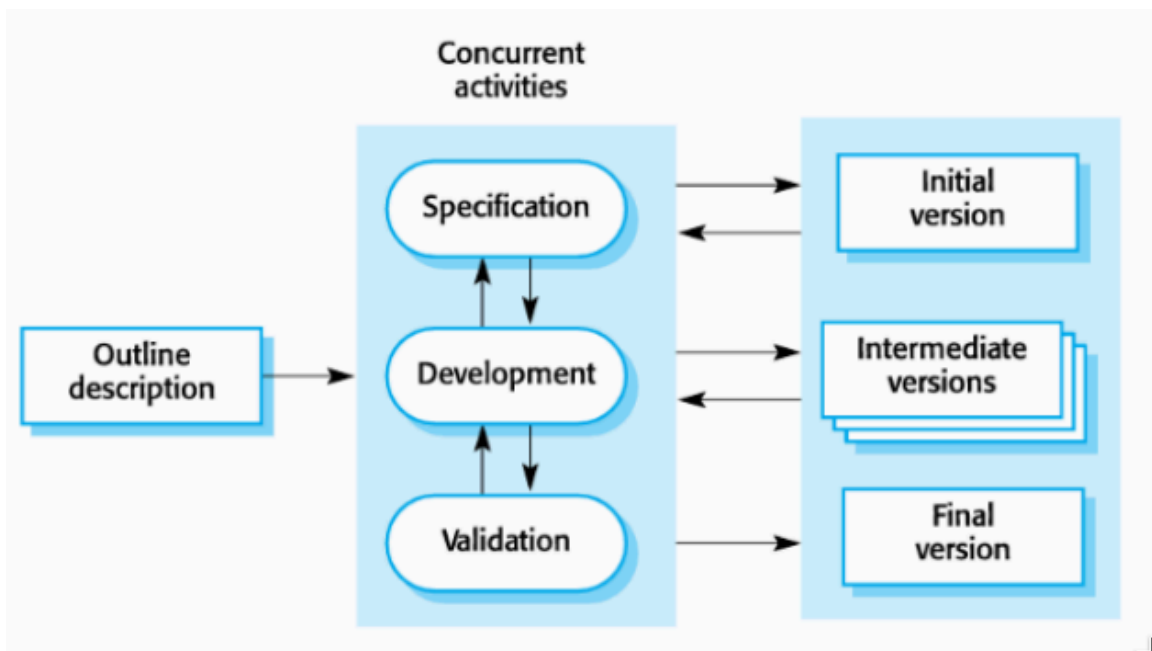
- **Disadvantages:**

- Difficult to accommodate changing customer requirements.
- A phase must be completed before moving to the next.
- Not suitable for projects where requirements change frequently.
- Customer feedback comes late in the development process.
- Errors made in early stages are expensive to fix later.

Incremental Development Model

- The Incremental Development Model develops the software in small parts called **increments**.
- Specification, development, and validation are carried out together.
- Each increment adds new functionality to the system.
- It can follow either a plan-driven or an agile approach.
- **Working / Stages:**
 - Start with the overall system requirements.

- Select a small set of requirements for the first increment.
- Carry out specification, development, and validation for that increment.
- Deliver the working increment to the customer.
- Collect customer feedback.
- Repeat the process for the next increment by adding more features.
- Continue until the complete system is developed.



- **Benefits:**

- Reduces the cost of accommodating changing customer requirements.
- Requires less rework of analysis and documentation than the waterfall model.
- Makes it easier to get customer feedback through demonstrations of working software.
- Allows rapid delivery and deployment of useful software.
- Customers can start using the software and gain value earlier.

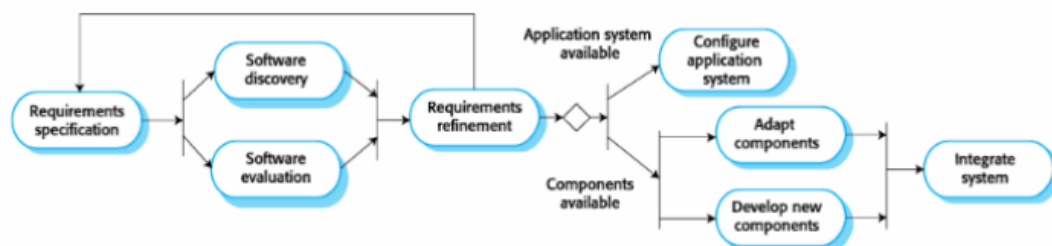
- **Problems:**

- The development process is less visible to managers.
- Producing documentation for every version may not be cost-effective.
- The system structure may degrade as new increments are added.

- Without regular refactoring, future changes become more difficult and costly.

Integration and Configuration (Reuse-Oriented) Model

- Based on **software reuse**, where systems are built using existing components or Commercial Off-The-Shelf (COTS) systems.
- Existing components can be configured to meet the user's requirements.
- Can be plan-driven or agile.
- Reuse is the standard approach for building many business systems.
- **Stages / Working:**
 - Requirements specification
 - Define the essential and desirable requirements.
 - Software discovery and evaluation
 - Search for suitable reusable components and evaluate them.
 - Requirements refinement
 - Modify requirements based on the available components.
 - Application system configuration
 - Configure off-the-shelf systems to meet user needs.
 - Component adaptation and integration
 - Adapt existing components if needed.
 - Develop new components when required.
 - Integrate all components into the complete system.



- **Types of reusable software:**

- Stand-alone application systems (COTS) configured for a specific environment.
 - Component packages integrated with frameworks such as .NET or J2EE.
 - Web services available for remote use.
 - **Advantages:**
 - Reduced development effort.
 - Lower development cost and risk.
 - Faster software delivery.
 - **Disadvantages:**
 - Requirements may have to be changed to fit available components.
 - Less control over the evolution of reused components.
 - Finding suitable reusable components can be difficult.
-

Process Activities

- Process activities are the main tasks carried out during software development.
- Every software process includes four fundamental activities.

1. Software Specification

- Define what the system should do.
- Identify customer requirements and constraints.
- Produce the software specification.

2. Software Development (Design and Implementation)

- Design the structure and organization of the system.
- Write the program code.
- Build the software according to the specification.

3. Software Validation

- Check that the software meets customer requirements.
- Test the software to ensure it works correctly.
- Verify that it performs the required functions.

4. Software Evolution

- Modify the software after deployment.
 - Adapt it to changing customer needs and market requirements.
 - Fix defects and add new features to keep the system useful.
 - These four activities are common to all software process models, although they may be organized differently in different processes.
-

Coping with Change

- Change is inevitable in all large software projects.
- System requirements change because of:
 - Business changes and new requirements.
 - New technologies.
 - Changing platforms.
- Change increases development cost because it causes rework such as re-analyzing requirements and implementing new functionality.
- **Two approaches to reduce the cost of change:**
 - **Change anticipation**
 - Includes activities that predict possible changes before major rework is needed.
 - A prototype may be developed to show key features to customers and refine requirements early.
 - **Change tolerance**
 - Designs the process so changes can be accommodated at low cost.
 - Usually achieved through incremental development.
 - Changes can be implemented in future increments or by modifying only a small part of the system.
- **Ways of coping with changing requirements:**
 - **System prototyping**

- A quick version of the system is developed to check customer requirements and design feasibility.
 - Supports change anticipation.
 - Helps users experiment with the system and refine requirements.
 - **Incremental delivery**
 - The system is delivered in small increments.
 - Customers provide feedback after each increment.
 - Supports both change anticipation and change tolerance.
 - Allows changes to be incorporated into later increments at lower cost.
 - **Benefits of prototyping**
 - Improves system usability.
 - Gives a closer match to users' real needs.
 - Improves design quality.
 - Improves maintainability.
 - Reduces development effort.
-

Agile Software Development

- Agile software development is an approach where **specification, design, implementation, and testing are carried out together**.
- It focuses on **rapid delivery of working software** and **responding quickly to changing customer requirements**.
- Planning is incremental, making it easier to accommodate changes during development.
- Agile methods reduce process overhead by minimizing unnecessary documentation.
- **Agile Manifesto values:**
 - Individuals and interactions over processes and tools.
 - Working software over comprehensive documentation.
 - Customer collaboration over contract negotiation.

- Responding to change over following a plan.
 - **Principles of agile methods:**
 - **Customer involvement**
 - Customers are closely involved throughout development.
 - They provide, prioritize, and evaluate requirements.
 - **Incremental delivery**
 - Software is developed and delivered in increments.
 - Customers decide the requirements for each increment.
 - **People, not process**
 - Team skills and collaboration are emphasized.
 - Developers are given flexibility in how they work.
 - **Embrace change**
 - Changes in requirements are expected and accommodated.
 - **Maintain simplicity**
 - Keep both the software and development process as simple as possible.
 - Eliminate unnecessary complexity.
 - **Advantages:**
 - Faster delivery of useful software.
 - Better response to changing requirements.
 - Continuous customer feedback.
 - Reduced development overhead.
 - Frequent releases improve customer satisfaction.
 - **Limitations:**
 - Requires active customer participation.
 - Can be difficult with large development teams.
 - Prioritizing changes from multiple stakeholders may be challenging.
 - Long-term planning and documentation may be limited.
-

Agile Development Techniques

- Agile development techniques are practices used to develop software quickly, improve quality, and adapt to changing requirements.
- A major agile method that introduced these techniques is **Extreme Programming (XP)**.
- **User Stories**
 - User stories are short descriptions of system features written from the user's perspective.
 - They replace long formal requirement documents.
 - They are created jointly by customers and developers.
 - They are written on story cards.
 - They are divided into small implementation tasks.
 - They help plan development iterations.
 - **Benefits:**
 - Easy for users to understand.
 - Encourage customer involvement.
 - Support quick delivery.
 - **Limitations:**
 - Difficult to ensure complete requirements.
 - Important details may be missed.
- **Refactoring**
 - Refactoring is the process of improving the structure of code without changing its external behavior.
 - It keeps the code simple and readable.
 - It prevents poor design caused by repeated changes.
 - It improves maintainability.
 - **Examples:**
 - Removing duplicate code.
 - Renaming variables and methods.

- Reorganizing class structure.
- **Limitation:**
 - Some major changes may require redesign instead of refactoring.
- **Test-First Development**
 - In test-first development, tests are written before the actual program code.
 - Automated testing is a central part of the process.
 - Tests are derived from user stories.
 - Development proceeds only when the tests pass.
 - Customers help define acceptance tests.
 - **Benefits:**
 - Clarifies requirements.
 - Detects errors early.
 - Reduces misunderstandings.
 - **Challenges:**
 - Some parts, such as user interfaces, are difficult to test.
 - Tests may not cover every possible case.
- **Pair Programming**
 - Pair programming is a practice where two programmers work together on one computer.
 - One programmer writes the code.
 - The other reviews the code and suggests improvements.
 - The programmers may switch roles regularly.
 - **Advantages:**
 - Improves code quality.
 - Encourages knowledge sharing.
 - Provides continuous code review.
 - Reduces the risk if team members leave the project.
 - **Concern:**

- It may appear less productive initially, but it improves long-term software quality.
 - These techniques support:
 - Frequent software releases.
 - Continuous improvement.
 - Better code quality.
 - Strong customer involvement.
 - Quick response to changing requirements.
-

Extreme Programming (XP)

- Extreme Programming (XP) is one of the most popular **Agile software development methods**.
- It was developed by **Kent Beck** in the late 1990s.
- XP focuses on rapid software development, continuous testing, customer involvement, and quick response to changing requirements.
- It follows an **iterative and incremental** approach where software is delivered in small releases.



The XP release cycle

- **Main Features of XP**
 - Small and frequent releases.

- Continuous customer involvement.
- Continuous testing.
- Pair programming.
- Refactoring.
- Continuous integration.
- Quick response to changing requirements.
- **Practices of XP**
 - **Incremental Planning**
 - Requirements are written as user stories.
 - Customers prioritize the stories.
 - Developers divide them into small implementation tasks.
 - Supports easy planning and changing requirements.
 - **Small Releases**
 - Software is developed and delivered in small, frequent releases.
 - Enables early delivery of useful software.
 - Provides faster customer feedback and reduces project risk.
 - **Simple Design**
 - Only the design needed for current requirements is developed.
 - Reduces complexity, simplifies maintenance, and speeds up development.
 - **Test-First Development**
 - Tests are written before the actual code.
 - Helps detect errors early, improves software quality, and clarifies requirements.
 - **Refactoring**
 - Improves the structure of code without changing its functionality.
 - Examples include removing duplicate code, renaming variables, and improving class structure.

- Increases readability, maintainability, and code quality.
 - **Pair Programming**
 - Two programmers work together on one computer.
 - One writes code while the other reviews and suggests improvements.
 - Results in fewer errors, better knowledge sharing, and improved software quality.
 - **Collective Ownership**
 - Any developer can modify any part of the code.
 - Promotes shared responsibility, faster problem solving, and teamwork.
 - **Continuous Integration**
 - New code is integrated into the system as soon as it is completed.
 - All tests are run after integration.
 - Helps detect defects early and reduces integration problems.
 - **Sustainable Pace**
 - Developers work at a pace that can be maintained over a long period.
 - Avoids excessive overtime, reducing stress and improving productivity.
 - **On-Site Customer**
 - A customer representative works closely with the development team throughout the project.
 - Provides immediate clarification of requirements and continuous feedback, leading to higher customer satisfaction.
 - **Advantages of XP**
 - Produces high-quality software.
 - Enables faster development and delivery.
 - Improves customer satisfaction.
 - Handles changing requirements easily.
 - Enhances teamwork and communication.
 - Detects defects early through continuous testing and integration.
-

Agile Project Management

- Agile project management is a way of managing software projects that supports **incremental development** and **agile practices**.
- Its main goal is to deliver software **on time** and **within the planned budget**.
- Unlike traditional plan-driven management, it adapts to changing requirements and frequent releases.
- **Traditional (Plan-Driven) Project Management**
 - Managers prepare a detailed project plan in advance.
 - The plan specifies:
 - What will be delivered.
 - When it will be delivered.
 - Who will work on each task.
 - Progress is measured against this predefined plan.
- **Agile Project Management**
 - Supports iterative and incremental development.
 - Plans are updated as the project evolves.
 - Focuses on frequent delivery of working software.
 - Encourages continuous customer feedback and collaboration.
 - Teams organize their work to respond quickly to changes.
- **Scrum**
 - Scrum is an agile project management method that focuses on managing iterative development.
 - It organizes development into fixed time periods called **sprints**.
- **Phases of Scrum**
 - **Initial planning phase**
 - Establish the project's general objectives.
 - Design the overall software architecture.
 - **Sprint cycles**

- Each sprint develops a working increment of the system.
 - Increments are reviewed and improved in subsequent sprints.
 - **Project closure phase**
 - Complete the project.
 - Finalize required documentation such as user manuals.
 - Review lessons learned from the project.
 - **Key Scrum terms**
 - **Development Team**
 - A self-organizing team responsible for developing the software and related documents.
 - **Product Backlog**
 - A prioritized list of work items, requirements, and tasks to be completed.
 - **Product Owner**
 - Identifies and prioritizes product features and continuously updates the backlog.
 - **Potentially Shippable Product Increment**
 - The software produced at the end of a sprint that is intended to be ready for release.
-

Scaling Agile Methods

- Scaling agile methods means adapting agile practices for **large projects, large teams, or large organizations**.
- Agile methods are most successful for small and medium-sized projects developed by small co-located teams.
- Scaling is needed when projects involve multiple development teams or teams working in different locations.
- **Two types of scaling:**
 - **Scaling up**
 - Applying agile methods to develop large software systems that cannot be handled by a single small team.

- **Scaling out**
 - Introducing agile methods across a large organization with many years of software development experience.
- **When scaling agile, it is important to maintain:**
 - Flexible planning.
 - Frequent software releases.
 - Continuous integration.
 - Test-driven development.
 - Good communication between teams.
- **Challenges in scaling agile:**
 - A completely incremental approach to requirements engineering is not possible.
 - There cannot be a single product owner or customer representative.
 - Large systems require some upfront design and documentation.
 - Cross-team communication mechanisms must be established.
 - Continuous integration becomes difficult, but frequent builds and regular releases should be maintained.
- **Practical problems with agile methods:**
 - Informal agile practices may conflict with legal contracts in large organizations.
 - Agile is more suitable for new software development than maintenance.
 - Agile assumes small co-located teams, while many real projects use distributed teams across different locations.
- **Factors affecting the choice between agile and plan-driven methods:**
 - Size of the system.
 - Type of system (e.g., safety-critical or real-time).
 - Expected system lifetime.
 - External regulations.
 - Skills of the development team.

- Team organization and distribution.
 - Availability of development tools.
 - Organizational culture and contractual requirements.
 - **Large organizations often use a combination of agile and plan-driven approaches** to balance flexibility with documentation, coordination, and long-term planning.
-